

[illegible][illegible]

■■■■ ■■■■■■■■■■: 9999 | Swagger UI: <http://localhost:9999/swagger-ui/index.html>

1. Project Overview

Goodoc — REST API backend, built with Spring Boot. It's a simple, clean, and easy-to-use API for managing documents. The project is designed to be a good example of a REST API backend, with a focus on simplicity and ease of use. It's a good starting point for anyone looking to build a REST API backend with Spring Boot.

The API is built with Spring Boot and uses JWT for authentication. It's a simple, clean, and easy-to-use API for managing documents. The project is designed to be a good example of a REST API backend, with a focus on simplicity and ease of use. It's a good starting point for anyone looking to build a REST API backend with Spring Boot.

The API is built with Spring Boot and uses JWT for authentication. It's a simple, clean, and easy-to-use API for managing documents. The project is designed to be a good example of a REST API backend, with a focus on simplicity and ease of use. It's a good starting point for anyone looking to build a REST API backend with Spring Boot.

2. Project Structure

Module	Version	Description
Java	17	Java 17
Spring Boot	3.3.4	Spring Boot 3.3.4
Spring Security	6.3.1	Spring Security 6.3.1
Spring Data JPA	3.3.1	Spring Data JPA 3.3.1
PostgreSQL	14+	PostgreSQL 14+
JWT (jjwt)	0.11.5	JSON Web Token 0.11.5
Lombok	-	Lombok 1.18.30
SpringDoc OpenAPI	2.0.2	SpringDoc OpenAPI 2.0.2
Maven	-	Maven 3.9.6

3. Project Architecture

The project is built using a layered architecture (Layered) design. The layers are as follows:

Client (Postman / Frontend / Mobile)

■

▼

Controller Layer ← REST-Controllers (@RestController)

■

▼

Service Layer ← Interfaces-Implementations (interface + impl)

■

▼

Repository Layer ← Spring Data JPA (■■■■■■■■■■)

■

▼

PostgreSQL Database

■■■■■■■■■■ ■■■■■■■■:

com.example.goodoc

■■■ config/ – SecurityFilter, JwtFilter, JwtService, ApplicationConfig, OpenAPIConfig

■■■ controller/ – AuthController, MedicineController, ReceptionController,

■ ReleaseFormController, UserController

■■■ dto/

■ ■■■ auth/ – AuthLoginRequest, AuthRegisterRequest, AuthResponse

■ ■■■ medicine/ – MedicineRequest, MedicineResponse

■ ■■■ reception/ – ReceptionRequest, ReceptionResponse

■ ■■■ releaseForm/ – ReleaseFormRequest, ReleaseFormResponse

■ ■■■ user/ – UserRequest, UserResponse

■■■ enums/ – Role (ADMIN, USER)

■■■ exception/ – CustomException, GlobalException, ApiResponse

■■■ mapper/ – ■■■■■■■■ ■■■■■■■■ + impl/

■■■ model/ – Medicine, ReleaseForm, Reception, User

■■■ repository/ – MedicineRepository, ReceptionRepository,

■ ReleaseFormRepository, UserRepository

■■■ service/ – ■■■■■■■■ ■■■■■■■■ + impl/

4. ■■■■■■ ■■■■■■ (Entity)

4.1 Medicine — ■■■■■■

名前	型	制約
id	Long	主キー (AUTO INCREMENT)
name	String	ユニーク
releaseForm	ReleaseForm	ManyToOne (ManyToOne → ReleaseForm)
reception	Reception	ManyToOne (ManyToOne → Reception)
nextDose	LocalDate	日付
time	LocalTime	時刻
additionally	String	追加情報

4.2 ReleaseForm — ■■■■■ ■■■■■■■■

■ ■ ■ ■	■ ■ ■	■ ■ ■ ■ ■ ■ ■ ■
id	Long	■■■■■■■■■■ ■■■■ (AUTO INCREMENT)
name	String (UNIQUE)	■■■■■■■■■■ ■■■■ (■■■■■■■■■■, ■■■■ ...)
medicines	List	■■■■■■■■■■ ■■■■■■■■■■ (OneToMany)

4.3 Reception — ■■■■■■ ■■■■■■

主键	数据类型	其他属性
id	Long	自增 (AUTO INCREMENT)
name	String (UNIQUE)	非空 (NOT NULL, ...)
medicineList	List	一对多 (OneToMany)

4.4 User —

■■■■	■■■	■■■■■■■■■■
id	Long	■■■■■■■■■■ ■■■■ (AUTO INCREMENT)
number	String	■■■■■ ■■■■■■■■ (■■■■■■■■■■■■■■■■■■■■ username)
password	String	BCrypt-■■■ ■■■■■■
role	Role	■■■■: ADMIN ■■■ USER

■ ■ ■ ■ ■: Medicine →ManyToOne→ ReleaseForm | Medicine →ManyToOne→ Reception | ReleaseForm
→OneToMany→ Medicine | Reception →OneToMany→ Medicine

5. ■■■■■■■■■■ — JWT + Spring Security

```

#####  #####  Spring Security  ■  #####  ■  JWT (JSON Web Token)  ###
stateless-#####.  ###  #####  #####  JwtFilter  #####
#####  Authorization,  #####  #####  ■  #####  #####
#####.

```

1	POST /auth/login POST /auth/register
2	, JWT (24)
3	{ "token": "..."}
4	: Authorization: Bearer <token>
5	JwtFilter
6	

■■■■■■■■ ■■■■■■■■■■ (■■■■ ■■■■■■■■):

```
POST /auth/register
POST /auth/login
GET /v3/api-docs/** (OpenAPI JSON)
GET /swagger-ui/** (Swagger UI)
```

■■■■ ■■■■■■■■■■■■ ■■■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ JWT-■■■■■.

6. REST API —

6.1 ■■■■■■■■■■ — /auth

Method	URL	Response
POST	/auth/register	{ "token": " "refresh_token": " }
POST	/auth/login	{ "token": "JWT-" }

POST /auth/register — ■■■■ ■■■■■■■■:

```
{
  "number": "77001234567",
  "password": "secret123",
  "confirmPassword": "secret123"
}
```

POST /auth/login — ■■■■ ■■■■■■■■:

```
{
  "number": "77001234567",
  "password": "secret123"
}
```

_____. _____.

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJyY2x1IjoiaW..."}

```

6.2 ■■■■■■■■ — /medicine

Method	URL	Description
GET	/medicine/all	Get all medicines
GET	/medicine/findById/{id}	Get medicine by ID
POST	/medicine/addMedicine	Add new medicine
PUT	/medicine/updateById/{id}	Update medicine by ID
DELETE	/medicine/deleteById/{id}	Delete medicine by ID

POST /medicine/addMedicine — ■■■■ ■■■■■■■■:

```
{
  "name": "■■■■■■■■■■",
  "releaseForm": "■■■■■■■■", ← ■■■■■■■■ ■■■■■■ ■■■■■■■■■■■■ ■ /releaseForm
```

```
{
  "reception": "■■■■■ ■■■", ← ■■■■■■■■ ■■■■■■ ■■■■■■■■■■ ■ /reception
  "month": "APRIL",
  "day": 15,
  "year": 2025,
  "hour": 8,
  "minute": 30,
  "timePeriod": "AM",
  "additionally": "■■■■■■■■ ■■■■■"
}
```

GET /medicine/all — ■■■■■■ ■■■■■■:

```
[
{
  "name": "██████████████████",
  "releaseForm": "██████████",
  "reception": "██████ █████",
  "nextDose": "2025-04-15",
  "time": "08:30:00",
  "additionally": "██████████ ██████"
}
]
```

6.3 ■■■■■ ■■■■■■ — /releaseForm

Method	URL	Request Body
GET	/releaseForm/all	
GET	/releaseForm/findById/{id}	{id}
POST	/releaseForm/create	{releaseForm}
PUT	/releaseForm/updateById/{id}	{releaseForm}
DELETE	/releaseForm/deleteById/{id}	{id}

POST /releaseForm/create — ■■■■ ■■■■■■■■:

```
{ "name": "██████████" }
```

GET /releaseForm/all — ■■■■■■ ■■■■■■:

```
[
  { "id": 1, "name": "██████████" },
  { "id": 2, "name": "██████" }
]
```

6.4 ■■■■■■ ■■■■■■ — /reception

Method	URL	Response
GET	/reception/all	[[{"id": 1, "name": "John", "age": 25}, {"id": 2, "name": "Jane", "age": 30}, {"id": 3, "name": "Bob", "age": 35}, {"id": 4, "name": "Alice", "age": 40}, {"id": 5, "name": "Charlie", "age": 45}]]
GET	/reception/findById/{id}	{ "id": 1, "name": "John", "age": 25 }
POST	/reception/create	{ "id": 6, "name": "David", "age": 50 }
PUT	/reception/updateById/{id}	{ "id": 1, "name": "John", "age": 26 }
DELETE	/reception/deleteById/{id}	{ "id": 1, "name": "John", "age": 25 }

POST /reception/create — ■■■■ ■■■■■■■■:

```
{ "name": "██████  █████" }
```

GET /reception/all — ■■■■■■ ■■■■■■:

```
[
{ "id": 1, "name": "██████ ███" },
{ "id": 2, "name": "██████" }
]
```

6.5 ■■■■■■■■■■ — /user

ADMIN.

□□□□□	□□□□□□□	□□□□□□□
-------	---------	---------

GET	/user/all	■■■■■■■■■■ ■■■■■■■■■■ ■■■■ ■■■■■■■■■■■■■■■■■■■■
GET	/user/findById/{id}	■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■■■ ID
DELETE	/user/deleteById/{id}	■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■■■ ID

7. ■■■■■■ ■■■■■■

```

GlobalException
(@ControllerAdvice)

```

```
{
  "message": "Medicine not found!",
  "status": 404
}
```

[illegible]

8. ■■■■■■

■ ■■■■■■ ■■■■■■■■■■ ■■■■-■■■■ ■■■ ■■■■ ■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■ Spring MockMvc ■ Spring Security Test.

■■■■ ■■■■■	■■■ ■■■■■■■■■■
MedicineControllerTest	CRUD ■■■■■■■■ ■■■ ■■■■■■■■■■■■ ■■■■■ MockMvc
ReceptionControllerTest	CRUD ■■■■■■■■ ■■■ ■■■■■■■■■■ ■■■■■■
ReleaseFormControllerTest	CRUD ■■■■■■■■ ■■■ ■■■■■■■■ ■■■■■■■■
UserControllerTest	■■■■■■■■ ■■■■■■ ■ ■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■
GoodocApplicationTests	■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■■■■■ ■■■ ■■■■■■

■■■■■■■ ■■■■ ■■■■■■■■:

```
./mvnw test
```

9. ■■■ ■■■■■■■■ ■■■■■

1. PostgreSQL

```
CREATE DATABASE goodoc_db;
```

2. application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/goodoc_db
spring.datasource.username=postgres
spring.datasource.password=██████████
spring.jpa.hibernate.ddl-auto=create ← ██████████ ██████████
```

3. How to use the data

```
./mvnw spring-boot:run
```

4. Swagger UI

`http://localhost:9999/swagger-ui/index.html`

spring.jpa.hibernate.ddl-auto=create → update, 数据库表结构

10. ■■■■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■■■■■■

■■■■■	■■■■■■■■■	■■■■■■■■■
POST	/auth/register	■■■■■■■■■■■■■■■
POST	/auth/login	■■■■■ / ■■■■■■■■■■ ■■■■■■■■
GET	/medicine/all	■■■■ ■■■■■■■■■■
GET	/medicine/findById/{id}	■■■■■■■■■■■■■■■ ■■■ ID
POST	/medicine/addMedicine	■■■■■■■■■■ ■■■■■■■■■■
PUT	/medicine/updateById/{id}	■■■■■■■■■■ ■■■■■■■■■■
DELETE	/medicine/deleteById/{id}	■■■■■■■■■■ ■■■■■■■■■■
GET	/releaseForm/all	■■■■ ■■■■■ ■■■■■■■■
GET	/releaseForm/findById/{id}	■■■■■■ ■■■■■■■■■ ■■■ ID
POST	/releaseForm/create	■■■■■■■■ ■■■■■ ■■■■■■■■
PUT	/releaseForm/updateById/{id}	■■■■■■■■ ■■■■■ ■■■■■■■■
DELETE	/releaseForm/deleteById/{id}	■■■■■■■■ ■■■■■ ■■■■■■■■
GET	/reception/all	■■■■ ■■■■■■■■■ ■■■■■■■■
GET	/reception/findById/{id}	■■■■■■■■ ■■■■■■■■■ ■■■ ID
POST	/reception/create	■■■■■■■■ ■■■■■ ■■■■■■■■
PUT	/reception/updateById/{id}	■■■■■■■■ ■■■■■ ■■■■■■■■
DELETE	/reception/deleteById/{id}	■■■■■■■■ ■■■■■ ■■■■■■■■
GET	/user/all	■■■■ ■■■■■■■■■■■■■■■■ (ADMIN)
GET	/user/findById/{id}	■■■■■■■■■■■■■■■■ ■■■ ID (ADMIN)
DELETE	/user/deleteById/{id}	■■■■■■■■ ■■■■■■■■■■■■■■■■ (ADMIN)